Chat-Protokoll

Technische Spezifikation

Version 1.0 · 2026

Praktikumsgruppe 1 – HAW Hamburg

Niklas · Lewin · Lukas · Hendrik · Ben · Juri · Vincent · Timo · Cedric

1 Überblick (Cedric) 2
2 Paketstruktur 2
2.1 Header (32 Byte, fix) (Cedric) 2
2.2 Payload-Segmentierung - (Vincent) 3
3 Pakettypen - (Vincent) 4
3.1 Dateiübertragung – Payload-Format - (Vincent) 4
4 Routing 5
4.1 Algorithmus: Distance-Vector-Routing (Ben) 5
4.2 Lokale Routingtabelle (Ben) 5
4.3 Verbindungsaufbau (Hendrik) 6
4.4 Regelmäßiger Tabellenaustausch (Cedric) 6
4.5 Payload-Format des ROUTING-Pakets (Juri) 6
4.6 Schleifenverhinderung (Juri) 6
4.7 Verbindungsabbau (Lewin) 7
5 Bestätigung und Fehlerbehandlung 7
5.1 ACK-Mechanismus (Lukas) 7
5.2 Heartbeat (Lukas) 7
5.3 Ablauf: Nachricht und Datei senden (Timo) 8
5.4 Senderate (Timo) 8

1 Überblick (Cedric)

Dieses Dokument beschreibt das binäre Netzwerkprotokoll für ein dezentrales Chat-System über UDP auf IPv4-Netzen. Es definiert den Paketaufbau, alle Pakettypen, das Routing-Verfahren sowie Regeln für Verbindungsauf- und -abbau.

Parameter	Wert
Transport	UDP
Netzwerk	IPv4
Port (Sender & Empfänger)	9999
Routingtabellen Port	9998
TTL (Startwert)	16 Hops
Max. Payload	1300 Byte
Kodierung (Text & Dateiname)	UTF-8
Kodierung (Dateidaten)	byte
Byte-Reihenfolge	Big Endian (Network Byte Order)

2 Paketstruktur

2.1 Header (32 Byte, fix) (Cedric)

Jedes Paket beginnt mit einem fixen 32-Byte-Header. Alle Felder werden in Big-Endian-Reihenfolge übertragen.

```
Offset  Größe  Feld
──────────────────────────────────────────────────
     0   1 Byte  Type (Pakettyp)
     1   1 Byte  TTL
     2   2 Byte  Nachrichten-ID
     4   4 Byte  Quelle (IPv4-Adresse)
     8   4 Byte  Ziel   (IPv4-Adresse)
    12   4 Byte  Payload-Länge
    16   8 Byte  Paketnummer
    24   8 Byte  Gesamt-Paketanzahl
──────────────────────────────────────────────────
Gesamt: 32 Byte

Max. Payload = 1500 (MTU) - 60 (IPv4) - 8 (UDP) - 32 (Header) - 100 (Puffer) =
1300 Byte
```

Feld	Offset	Größe	Beschreibung
Type	0	1 Byte	Pakettyp als Hex-Wert (siehe Abschnitt 3).
TTL	1	1 Byte	Time-To-Live. Wird bei jedem Hop um 1 verringert. Bei TTL = 0 wird das Paket verworfen (bzw. nicht weitergeleitet). Startwert: 16.
Nachrichten-ID	2	2 Byte	Eindeutige, monoton steigende Nummer pro Absender (uint16). Overflow: 65535 → 0. Beginnt bei 0.
Quelle	4	4 Byte	IPv4-Adresse des Absenders (32-Bit-Integer, Network Byte Order).
Ziel	8	4 Byte	IPv4-Adresse des Empfängers.
Payload-Länge	12	4 Byte	Anzahl der Nutzdaten-Bytes hinter dem Header. 0 = kein Payload (z. B. ACK).
Paketnummer	16	8 Byte	Position dieses Chunks innerhalb der aufgeteilten Nachricht. Beginnt bei 0.
Gesamt-Paketanzahl	24	8 Byte	Gesamtzahl der Chunks dieser Nachricht. Ermöglicht dem Empfänger, Speicher vorab zu reservieren.

Keine Prüfsumme im Header

Eine CRC32-Prüfsumme entfällt, da UDP bereits eine eigene Prüfsumme mitführt. Dadurch erreicht der Header exakt 32 Byte. Die Integrität wird vollständig durch die UDP-Checksum gewährleistet.

2.2 Payload-Segmentierung - (Vincent)

Textnachrichten (MSG) & Dateien (FILE), die 1300 Byte überschreiten, werden in Chunks aufgeteilt. Jeder Chunk wird als eigenständiges Paket mit eigenem Header gesendet. Paketnummer und Gesamt-Paketanzahl ermöglichen die Reassemblierung beim Empfänger.

- Pakete dürfen in beliebiger Reihenfolge ankommen und werden anhand der Paketnummer sortiert.
- Erst wenn alle Chunks einer Nachrichten-ID empfangen wurden, gilt die Dateiübertragung als vollständig.
- Bleibt ein ACK aus, wird das betroffene Paket erneut gesendet (siehe Abschnitt 5).

3 Pakettypen - (Vincent)

Der Type-Wert im Header (1 Byte) bestimmt die Semantik eines Pakets.

Typ-Name	Hex-Wert	Port	Rate Limit	Beschreibung
ROUTING	0x01	9998	Nein	Routingtabellen-Update. Payload enthält die serialisierte Routingtabelle (binär).
FILE	0x02	9999	Ja	Dateiübertragung. Chunk 0 enthält den Dateinamen (UTF-8), alle Chunks enthalten Dateidaten als Byte (Chunk 0: max. 1298 Bytes, Chunk (1...n) max. 1300 Bytes)
MSG	0x03	9999	Ja	Textnachricht. Payload enthält den UTF-8-kodierten Text.
ACK	0x04	9999	Nein	Empfangsbestätigung. Kein Payload (Payload-Länge = 0). Nachrichten-ID referenziert das bestätigte Paket.
BYE	0x7F	9998	Nein	Geordneter Verbindungsabbau. Kein Payload. Empfänger entfernt Absender aus seiner Routingtabelle.

3.1 Dateiübertragung – Payload-Format - (Vincent)

Der Dateiname wird ausschließlich im ersten Chunk (Paketnummer = 0) übertragen:

```
Chunk 0 (Paketnummer = 0):
+----------+-----------------+---------------------------------------+
| nameLen  |    Dateiname    |  Dateidaten (Anfang)    |
|  1 Byte  |  nameLen Byte   |  bis zu (1299 - nameLen) Byte         |
+----------+-----------------+---------------------------------------+

Chunk 1..n (Paketnummer = 1, 2, ...):
+-------------------------------------------------------------------+
|             Dateidaten, bis zu 1300 Byte                          |
+-------------------------------------------------------------------+

  nameLen  = vorzeichenloser 1-Byte-Integer (1-255)
  Dateiname = UTF-8-kodiert (Max 255 Byte)
```

4 Routing

4.1 Algorithmus: Distance-Vector-Routing (Ben)

Jeder Knoten kennt ausschließlich seine direkten Nachbarn und deren Routingtabellen. Routen zu weiter entfernten Knoten werden durch den zeitbasierten Austausch dieser Tabellen (Gossip-Verfahren) gelernt.

4.2 Lokale Routingtabelle (Ben)

Jeder Knoten pflegt intern eine Tabelle mit folgenden Feldern pro Eintrag:

Feld	Typ (Header) / Größe	Typ	Beschreibung
Name	char[], 1..32 Byte	String	Menschenlesbarer Name des Zielknotens (angezeigter Name).
ip	uint32, 4 Byte	IPv4	IP-Adresse des Zielknotens. Dient als eindeutige ID im Netz.
next_hop	uint32, 4 Byte	IPv4	IP des nächsten Hops auf dem Weg zum Ziel. Bei Direktverbindung gleich der Ziel-IP.
hops	uint8, 1 Byte	Hop-Anzahl	Metrik (Distanz). Direkt = 1, +1 pro weiterem Hop. Wert > 16 bedeutet unerreichbar.
last_seen	uint64, 8 Byte	Unix-Timestamp	Zeitpunkt der letzten Aktualisierung. Dient zur Timeout-Erkennung (Schwellwert: 30 Sek.).

Hinweis: ip = eindeutige ID des Knotens (maschinenlesbar), Name = Klarname (menschenlesbar). Beide Felder werden getrennt gepflegt.

4.3 Verbindungsaufbau (Hendrik)

Ein neuer Teilnehmer sendet sofort ein ROUTING-Paket (0x01) einzeln an jeden bekannten direkten Nachbarn (Unicast, Port 9998). Dieses Paket enthält ausschließlich den eigenen Eintrag (hops = 0). Es wird kein separates HELLO-Paket verwendet.

```
Neuer Knoten A                      Nachbar B
     |                                  |
     |--- ROUTING (nur A, hops=0) --->|  (Unicast an jeden Nachbarn)
     |                                  |  Tabelle aktualisieren
     |<-- ROUTING (B's Tabelle) ------|
     |                                  |
     |   Eigene Tabelle berechnen     |
     |                                  |
     |--- ROUTING (aktualisiert) ---->|  (Weiterpropagation)
     |                                  |
```

4.4 Regelmäßiger Tabellenaustausch (Cedric)

Routingtabellen werden ausschließlich per Unicast an direkte Nachbarn gesendet, niemals geflutet. Der Versand erfolgt zeitbasiert alle 5 Sekunden.

4.5 Payload-Format des ROUTING-Pakets (Juri)

Das ROUTING-Paket trägt eine binär serialisierte Liste von Routen-Einträgen als Payload.

```
Pro Routen-Eintrag (variabler Länge):
+----------+----------+-------------+---------+----------+
|    ip    |   hops   |  nameLen    |         name         |
|  4 Byte  |  1 Byte  |  1 Byte     |     nameLen Byte     |
+----------+----------+-------------+---------+----------+

  ip       = IPv4-Adresse des Zielknotens (Network Byte Order)
  hops     = Metrik (uint8)
  nameLen  = Länge des Klarnamens in Byte (1-32)
  name     = UTF-8-kodierter Klarname (variabel, max. 32 Byte)


Payload = [Eintrag_0][Eintrag_1]...[Eintrag_n]

Nicht übertragen: last_seen und next_hop.
  last_seen = Empfänger setzt aktuellen Zeitstempel.
  next_hop  = Empfänger setzt Absender-IP aus dem Paket-Header.
```

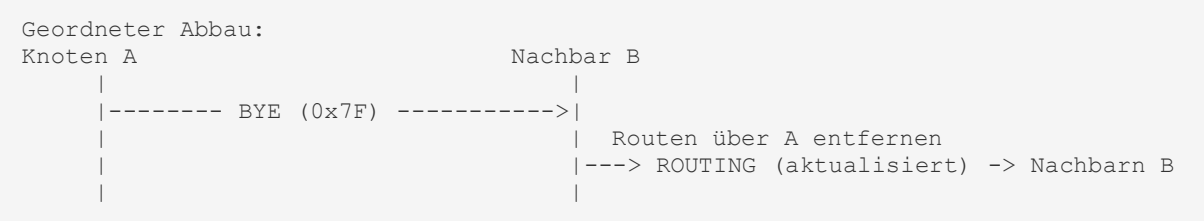
4.6 Schleifenverhinderung (Juri)

TTL: Jeder weiterleitende Knoten verringert den TTL-Wert um 1. Bei TTL = 0 wird das Paket verworfen.

4.7 Verbindungsabbau (Lewin)

Geordneter Abbau: Der Teilnehmer sendet ein BYE-Paket (0x7F) an alle direkten Nachbarn über den Routing-Port (9998). Die Nachbarn entfernen den Absender aus ihrer Routingtabelle und löschen alle Routen, die über diesen Knoten liefen. Anschließend propagieren sie mit dem Heartbeat ihre aktualisierte Tabelle.

Timeout (ungeordneter Abbau): Bleibt das periodische ROUTING-Paket eines Nachbarn länger als 30 Sekunden aus, gilt der Knoten als verloren. Alle betroffenen Routen werden entfernt und die Änderung propagiert. Routen mit Metrik > 16 gelten als unerreichbar und werden verworfen.



5 Bestätigung und Fehlerbehandlung

5.1 ACK-Mechanismus (Lukas)

Jedes empfangene MSG- und FILE-Paket wird mit einem ACK-Paket (0x04) bestätigt. Das ACK-Paket hat keinen Payload (Payload-Länge = 0). Die Nachrichten-ID im ACK-Header referenziert die ID des bestätigten Chunks.

- Pro Chunk wird genau ein ACK gesendet.
- ACK-Pakete werden selbst nicht bestätigt.
- Bleibt ein ACK nach einem definierten Timeout (1s) aus, sendet der Sender das betreffende Paket erneut.
- Wenn 10 Sekunden kein ACK für eine Nachrichten-ID kommt, werden keine Pakete mehr gesendet

5.2 Heartbeat (Lukas)

Da UDP keine Verbindung aufrechterhält, dienen die periodischen ROUTING-Pakete (alle 5 Sekunden) gleichzeitig als Heartbeat-Signal. Bleibt dieses Signal 30 Sekunden aus, gilt der Knoten als ausgefallen (siehe Abschnitt 4.7).

5.3 Ablauf: Nachricht und Datei senden (Timo)

```
Textnachricht (MSG, 1 Paket):
Sender A                          Empfänger B
     |                                 |
     |-- MSG [ID=42, Pkt=0, Ges=1] -->|
     |<-- ACK [ID=42, Pkt=0, Ges=1] --|
     |                                 |

Dateiübertragung (FILE, mehrere Chunks):
Sender A                          Empfänger B
     |-- FILE [ID=43, Pkt=0, Ges=3] ->|  (enthält Dateiname)
     |-- FILE [ID=43, Pkt=1, Ges=3] ->|
     |-- FILE [ID=43, Pkt=2, Ges=3] ->|
     |<-- ACK [ID=43, Pkt=0] ---------|
     |<-- ACK [ID=43, Pkt=1] ---------|
     |<-- ACK [ID=43, Pkt=2] ---------|

Retransmission bei ausbleibendem ACK:
     |-- FILE [ID=43, Pkt=1, Ges=3] ->|  (Timeout, 1. Wiederholung)
     |-- FILE [ID=43, Pkt=1, Ges=3] ->|  (Timeout, 2. Wiederholung)
     |                           max. 10s bis Abbruch

Ges = Gesamt-Paketanzahl
Pkt = Paketnummer
ID = Nachrichten-ID
```

5.4 Senderate (Timo)

Jeder Knoten sendet eigene Pakete mit einer festen maximalen Rate (1 Paket pro Millisekunde). Dies verhindert, dass einzelne Knoten das Netz überlasten, und sorgt für ein vorhersehbares Verhalten im vermaschten Netz.

- Die Rate gilt für alle selbst erzeugten Pakete (MSG, FILE-Chunks, ROUTING, BYE) zusammen.
- Weiterzuleitende Pakete (fremde Pakete, die über diesen Knoten geroutet werden) und ACK-Pakete fallen nicht unter diese Begrenzung.